Relatório Trabalho 3 de Arquitetura

Alunas: Maria Eduarda Prada da Silva e Isabella dos Santos Stuart

Sagui Vetorial

Foram utilizados os opcodes reservados 1110 e 1111 para implementar a instrução ADDI, Add Immediate, nos bancos escalar e vetorial. Essa instrução foi escolhida porque o programa precisava somar constantes pequenas aos registradores, principalmente para calcular endereços dentro do laço. Sem o ADDI, seria necessário carregar o imediato em outro registrador antes de somar, gastando mais instruções e registradores.

- s.addi, opcode 1110: soma um imediato de 4 bits estendido diretamente no registrador escalar de propósito geral SR1. Operação teórica: SR[1] = SR[1] + Imm.
- v.addi, opcode 1111: soma um imediato de 4 bits estendido diretamente no registrador vetorial de propósito geral VR1 de todos os PEs em paralelo. Operação teórica: VR[1] = VR[1] + Imm.

Com isso, o código ficou mais curto, economizou registradores e facilitou os deslocamentos entre A[i], B[i] e R[i] na memória.

Sinais da Unidade de Controle

A ordem elétrica dos bits de controle no Logisim foi mapeada da esquerda para a direita na seguinte estrutura (Bit 9 ao Bit 0): [ALUSrc, SelVPE, RegWrite, ALUOp(4 bits), MemRead, MemWrite, Branch].

Opcode	**Instrução**	**ALU Src**	**SelVPE**	**Reg Write**	**ALU Op**	**Mem Read**	**Mem Write**	**Branch**	**Hex (ROM)**
0000	s.ld	0	0	1	0000	1	0	0	**084**
0001	s.st	0	0	0	0000	0	1	0	**002**
0010	s.movh	1	0	1	1001	0	0	0	**2C8**
0011	s.movl	1	0	1	1000	0	0	0	**2C0**
0100	s.add	0	0	1	0000	0	0	0	**080**
0101	s.sub	0	0	1	0001	0	0	0	**088**
0110	s.and	0	0	1	0010	0	0	0	**090**

0111	s.brzr	0	0	0	0001	0	0	1	**009**
1000	v.ld	0	1	1	0000	1	0	0	**184**
1001	v.st	0	1	0	0000	0	1	0	**102**
1010	v.movh	1	1	1	1001	0	0	0	**3C8**
1011	v.movl	1	1	1	1000	0	0	0	**3C0**
1100	v.add	0	1	1	0000	0	0	0	**180**
1101	v.sub	0	1	1	0001	0	0	0	**188**
1110	s.addi	1	0	1	1010	0	0	0	**2D0**
1111	v.addi	1	1	1	1010	0	0	0	**3D0**

2. Tabela do Código Principal (Hex do assembly do Sagui Vetorial)

Carga Hexadecimal Contínua:
C8 99 B4 C9 B1 99 B4 C9 B2 99 BC C9 B3 99 B4 C9 B4 99 B4 C9 B5 99 DA B6 99 B7 99 B8 99 BF 99 33 20 5A 49 37 22 5F 4D BF 89 D5 C6 89 F3 8D CB F3 99 BF 89 B1 C9 BF 99 31 20 59 3E 23 79 73 71

O programa realiza a soma vetorial R[i] = A[i] + B[i] usando a RAM local de cada PE para contornar a limitação de registradores da arquitetura de 8 bits. Na inicialização, o VR2 é usado como acumulador para preencher os vetores A, B e R na memória, enquanto o índice inicial do laço, i = 0, é salvo no endereço 15. No SPE, os registradores escalares são separados por função: SR1 é usado apenas como auxiliar para carregar imediatos, SR2 guarda o contador do laço com valor inicial 3, e SR3 guarda o endereço de retorno 0x27, evitando que esses valores sejam sobrescritos por instruções movl e movh. Durante o laço, o índice salvo na RAM é lido, usado para acessar A[i], depois incrementado com deslocamento de 3 para acessar B[i], e novamente deslocado para gravar o resultado em R[i]. Ao final de cada iteração, o índice é atualizado na RAM e o contador em SR2 é decrementado; quando ele chega a zero, o programa desvia para 0x3E, onde permanece em loop infinito na mesma instrução, caso contrário retorna para 0x27. Ao final, a RAM local dos PEs confirma o funcionamento correto: PE0 possui A = 00 04 08, B = 14 18 1C e R = 14 1C 24; PE1 possui A = 01 05 09, B = 15 19 1D e R = 16 1E 26; PE2 possui A = 02 06 0A, B = 16 1A 1E e R = 18 20 28; e PE3 possui A = 03 07 0B, B = 17 1B 1F e R = 1A 22 2A.

Sagui VLIW

O datapath do processador de arquitetura VLIW funciona da seguinte maneira:

PC aponta para a Dual Port RAM, que aqui contém 32 bits de dados e 8 de endereçamento. O valor do PC é controlado por 1 MUX, que recebe PC + 1 na entrada 0 e um endereço de salto (cujo cálculo será explicado posteriormente) na entrada 1. Ele é controlado pelo sinal de controle PCSrc. O PC também possui um botão de reset. Na subida do clock, é lido o dado contido no endereço apontado (data), enquanto na descida é lida a instrução (instr). Ao longo da execução do trabalho, percebeu-se que essa ordem de leitura não funcionava adequadamente, especialmente para as instruções de salto. Logo, para essa implementação, o PC, a Memória de Instruções e o Banco de Registradores recebem o clock invertido por uma porta NOT.

A Dual Port RAM possui 5 entradas:
- addr: endereço de leitura ou escrita de dados, recebe como entrada o registrador rb.
- pc: posição atual do PC.
- d: dado a ser escrito na memória, recebe como entrada o registrador ra.
- w: sinal de escrita na memória, recebe o sinal de controle MemWr.
- clock.

Em seguida, a palavra passa por vários splitters, que a dividem em 4 instruções (4 lanes), cada uma com seu opcode e seus dados. Para cada uma das lanes, foi implementada uma unidade de controle própria, controlada por um circuito combinacional. Assim, os opcodes são enviados às unidades de controle e os bits de dados são enviados ao Banco de Registradores e/ou à ULA, quando conveniente. Os sinais de controle gerados funcionam de forma independente entre as lanes (por exemplo, o RegWr da lane 1 não é o mesmo que o da lane 3).
Sinais de controle gerados pelas unidades de controle:

Opcode	Instrução	MemWr	MemtoReg	Jump	RegWr	Branch	ALUSrc	ALUOp	Addz
0000'	brzr	0	0	0	0	1	0	00'	0
0001'	brzi	0	0	0	0	1	1	01'	0
0010'	jr	0	0	1	0	0	0	10'	0
0011'	ld	0	1	0	1	0	0	0	0
0100'	st	1	0	0	0	0	0	0	0
0101'	movh	0	0	0	1	0	1	0	0
0110'	movl	0	0	0	1	0	1	1	0
0111'	add	0	0	0	1	0	0	0	0
1000'	sub	0	0	0	1	0	0	1	0
1001'	**nova = addz**	0	0	0	1	0	0	0	1
1010'	**nova = addi**	0	0	0	1	0	1	0	0
1011'	or	0	0	0	1	0	0	001'	0
1100'	not	0	0	0	1	0	0	010'	0
1101'	slr	0	0	0	1	0	0	011'	0
1110'	srr	0	0	0	1	0	0	100'	0
1111'	nop	0	0	0	0	0	0	000'	0

As instruções executadas pelos opcodes seguem o padrão presente no conjunto de instruções SAGUI fornecido previamente. As duas novas instruções escolhidas foram:

- addi ra, imm: adiciona um imediato de 2 bits (valor inteiro entre 0 e 3) diretamente no registrador ra. Sua utilidade se deve ao fato de essa ser uma operação muito frequente em loops e também em contextos de somas de pequenos valores. Ela possibilita a fácil implementação de loops que realizam incrementos de 2 em 2 e de 3 em 3, por exemplo. Sem ela, seriam necessárias as instruções: zerar r0 -> movl imm -> add ra, r0.
- addz ra, rb: armazena a soma ra + rb em r0. Sua vantagem está no fato de que possibilita a realização de uma soma sem a perda dos dados de um dos registradores envolvidos na instrução. Isso é especialmente útil no algoritmo proposto no enunciado, pois possibilita que simultaneamente o programa armazene os endereços de R, A e B em registradores, facilitando significativamente as operações de load, soma e store requisitados. Com ela, é possível gerenciar o algoritmo, possibilitando a economia de vários ciclos de clock que seriam gastos para o cálculo de um endereço, por exemplo, que envolveria: zerar ra -> movl imm -> movh imm -> add ra, r0.

No banco de registradores, a lógica implementada foi a seguinte:
Para permitir a leitura e a escrita simultâneas em vários registradores, o banco de registradores foi projetado da seguinte maneira: ele possui 8 entradas e 8 saídas (2 entradas e 2 saídas por lane, para ra e rb), além de um botão de reset, como solicitado no enunciado.

Para o sistema de leitura dos registradores, foi implementado um mux em cada saída. As entradas de cada mux são os dados armazenados em cada registrador e a saída é o dado contido no registrador desejado. A lógica de controle do mux varia conforme a necessidade: para as lanes 2 e 4, é o próprio ra, enquanto para as lanes 1 e 3 foram tratados os casos em que o registrador de destino seria diferente de ra (para as instruções movh, movl e addz).

Já o sistema de escrita em registradores foi organizado da seguinte maneira: o sinal de write enable de cada registrador é controlado por uma porta OR de 3 entradas: os sinais “l1_r”, “l3_r”e “l4_r”. Para gerar esses sinais, foi implementado, para cada lane, um demux controlado pelo sinal regwrite correspondente, cuja chave de controle é o número do registrador de destino. Nesse caso, são somente 3 lanes envolvidas, uma vez que a lane 2 contém somente instruções de salto e, portanto, não envolve escrita em registradores. Enquanto isso, a entrada de dados recebe um mux, cujas entradas são os dados de cada lane (para as lanes 3 e 4, são o resultado da operação feita na ULA, enquanto para a lane 1 é uma mux entre um dado da memória e o resultado da ULA, controlado pelo sinal MemtoReg). Esse mux recebe 2 bits de controle, que equivalem ao dado que deve ser escrito no registrador. Para os bits de controle, temos 2 portas OR: a primeira, equivalente ao bit de controle mais significativo do mux, recebe como entrada os

sinais “l3_r” e “l4_r”, enquanto a segunda recebe “l1_r” e “l4_r”. Essa lógica funciona devido ao fato de considerarmos que todas as dependências de dados serão resolvidas via software e, dessa forma, não haverá 2 lanes tentando escrever no mesmo registrador simultaneamente. Assim, se queremos escrever um dado da lane 1, por exemplo, será ativado o sinal “l1_r”, que ativará somente o segundo OR, gerando um sinal “01” como controle para o mux, que selecionará o dado número 1. O mesmo funcionará para as outras lanes: o sinal “10” indica a escrita de um dado da lane 3 e o sinal “11” indica a de um dado da lane 4.
O circuito também possui uma ULA para cada lane. As ULAS 1 e 3 recebem como entradas o registrador ra e um MUX controlado pelo sinais ALUSrc correspondentes e possuem o resultado da operação executada e o sinal Zero como saída. A ULA 2 recebe somente ra como entrada (pois seu único objetivo é compará-lo a zero, para gerar os devidos sinais de salto) enquanto a ULA 4 recebe diretamente ra e rb como entradas.

O endereço de salto é gerado por um mux, que recebe rb na entrada 0 e PC + Imm na entrada 1. Esse mux é controlado pelo sinal de controle ALUSrc (para a instrução brzi). Já o sinal PCSrc é controlado por uma porta OR que recebe o sinal Jump e uma porta AND como entradas. Essa porta AND envolve os sinais Branch e Zero.

Em relação ao algoritmo de teste desenvolvido, a grande vantagem para sua eficiência é, justamente, o paralelismo proporcionado pela arquitetura VLIW. A lógica implementada envolveu 2 loops, um para a alocação dos vetores e outro para as somas R[i] = A[i] + B[i].
No primeiro, são calculados o endereço de R[i] e o tamanho dos vetores (12). Depois, armazena 0 em R[i], soma 12 a esse endereço, armazena o iterador em A[i] (pois, como o looping é crescente, o valor armazenado em cada posição de A sempre coincide com o valor do iterador), soma 12 ao endereço novamente, acrescenta 20 ao valor do iterador (o resultado é armazenado em um registrador diferente do utilizado para o iterador, obviamente) e esse valor é armazenado em B[i].
Já no segundo, seguindo o mesmo raciocínio do looping anterior, calcula-se os endereços de R[i], A[i] e B[i]. Para ambos os loopings, é feito o armazenamento dos 3 endereços simultaneamente. Com isso, basta fazer os loads, a soma e o store diretamente. Uma vantagem é que a arquitetura VLIW permite o paralelismo nessas operações: enquanto B[i] = A[i] + 12 é calculado, já é possível fazer o load de A[i], por exemplo. Isso diminui consideravelmente o número de ciclos de clock exigidos para a conclusão da execução do programa.
Para o fim do loop, é calculado um endereço de salto que leva a uma instrução de jr, repetindo o salto para o próprio endereço sucessivas vezes, o que simula a instrução “ebreak” do emulador EGG.